Flight Booking Website — Explanation (Part 2/5)

Part 2: Backend configuration, DB connection, CORS, and authentication endpoints.

1) Backend request/response style

Each file in `backend/api` is a stand-alone PHP script that: sets headers (usually JSON + CORS), reads inputs (JSON body or query string), talks to the DB via PDO, and echoes a JSON response.

Many endpoints follow this pattern:

```
header('Content-Type: application/json');
require_once '../config/database.php';
$db = (new Database())->getConnection();
// ... validate method and inputs ...
// ... run SQL via prepared statements ...
echo json_encode([...]);
```

2) Configuration: backend/config/config.php

Purpose: central “application config” file. It typically:

- Defines constants like app name/version, environment, JWT settings (if used).
- Sets CORS headers (Access-Control-Allow-Origin/Methods/Headers).
- Handles browser preflight requests (`OPTIONS`) by exiting early with 200.

Why CORS matters: your frontend pages are often served from a different origin/port than the API, so without CORS the browser blocks requests.

3) Database connection: backend/config/database.php

Purpose: create a PDO connection to MySQL/MariaDB. It normally contains host, dbname, username, password, and PDO options.

- **Key method:** `Database::getConnection()`
- **Output:** returns a PDO instance or null/throws (depends on implementation).
- **Used by:** almost every endpoint and model file.

4) Authentication approach in this repo

The “World A” pages use a simple token/session approach stored in the browser (usually `localStorage`). The backend endpoints commonly return a user object (id, user_type, etc.) after successful login.

Important: while `backend/config/config.php` mentions JWT constants, most “World A” endpoints do not actually validate JWTs on each request. Instead they typically trust a `user_id` passed in the request. That is OK for a class project, but it’s not secure for production.

5) Endpoint: backend/api/auth.php

Purpose: login endpoint.

- **Method:** usually `POST`
- **Input:** JSON body with email + password
- **DB action:** select the user record by email
- **Password check:** compare using password hashing utilities (for hashed passwords) or plain compare (depending on what the DB contains)
- **Output:** success/failure JSON; often includes user id, user_type, and sometimes a token.

If passwords were created with `password_hash`, then login must use `password_verify`. If they were inserted as plain text in seeds, password_verify will fail unless the seed matches hashing.

6) Endpoint: backend/api/register.php

Purpose: create new user accounts (company or passenger).

- **Method:** `POST`
- **Input:** JSON body (name/email/password/tel/user_type + optional fields)
- **DB action:** insert into `users`

- **Special behavior:** sets `account_balance` initial value depending on `user_type` in the endpoint logic.
- **Output:** JSON success + created user id (or error).

7) Profiles & password endpoints

These endpoints support viewing and updating user profile data:

- `backend/api/get-profile.php`: fetch a user record by id.
- `backend/api/update-profile.php`: update profile fields (company bio/logo, passenger photo/passport, etc.).
- `backend/api/change-password.php`: change password; typically validates current password then updates to new password.

The exact fields updated depend on whether the user is company or passenger.

8) Files covered in Part 2 (nothing skipped within this topic)

- `backend/config/config.php`
- `backend/config/database.php`
- `backend/api/auth.php` (+ any duplicate like `auth_new.php` if present)
- `backend/api/register.php`
- `backend/api/get-profile.php`, `update-profile.php`, `change-password.php`

End of Part 2/5.